



**Universidade Federal do Ceará
Campus Quixadá**

QXD0068 – Reuso de Software

Prof. Francisco Victor da Silva Pinheiro

Bruno da Silva Lustosa de Melo - 510709

Gabriel Alves Pessoa - 512131

Juan David Bizerra Pimentel- 557340

Lorena Emanuele Pinheiro Ferreira - 516293

Trabalho Final

Solução Integrada de Reuso / Auditoria e Melhoria de Sistema Legado

**Quixadá-CE
2025**

1. Introdução	3
2. Diagnóstico do Sistema	3
2.1. Evidências Métricas	4
3. Melhorias Propostas	5
4. Implementação	6
Comparativo de Métricas (Antes vs. Depois)	11
Métricas do Padrão Facade (Empréstimos)	11
Métricas da Refatoração do Subsistema de Relatórios (PDF)	12
Integração com PDF-Microservice	12
Benefícios obtidos em termos de reuso	12
Demonstração de reutilização em outro cenário	13
6. Discussão	13
7. Conclusão	14
8. Referências	15

1. Introdução

O projeto Bibliotech consiste em um sistema de gestão para bibliotecas escolares, abrangendo desde o controle de acervo e empréstimos até a geração de relatórios administrativos. Este trabalho, desenvolvido para a disciplina QXD0068 - Reuso de Software da UFC Quixadá, visa analisar a arquitetura atual do sistema sob a ótica da reusabilidade sistemática, identificando gargalos que impedem a evolução eficiente da plataforma.

A motivação para esta intervenção reside na necessidade de reduzir a redundância de código e o acoplamento rígido entre componentes. Através da aplicação de métricas técnicas e padrões de projeto (GoF), propõe-se uma transição para uma arquitetura modularizada que suporte a extração de ativos reutilizáveis, como o serviço de notificações e a renderização de documentos.

2. Diagnóstico do Sistema

Nesta etapa, o sistema foi avaliado utilizando métricas clássicas de engenharia de software para quantificar seu nível de reuso e modularidade:

- **Acoplamento Entre Objetos (CBO):**

O `EmprestimosService` apresenta um CBO de 6, possuindo dependências diretas de repositórios, especificações, mappers e serviços de notificação. Embora o uso do padrão Facade (`EmprestimoFacade`) já tenha sido iniciado para mitigar a complexidade, o service ainda concentra muitas responsabilidades de orquestração.

- **Falta de Coesão em Métodos (LCOM):**

A classe `PdfExportService` atua como uma God Class (Classe Deus). Embora delegue a criação para fábricas específicas (`FrequenciaAlunosPdfFactory`, `RelatorioAcervoPdfFactory`), ela ainda centraliza o conhecimento de todos os tipos de exportação do sistema, o que diminui sua coesão funcional.

- **Taxa de Duplicação de Código:**

Identificada alta duplicação na lógica de formatação de PDFs. A classe `PdfHeaderUtil` contém informações da escola (nome, endereço, CNPJ) "hardcoded". Isso impede o reuso do sistema em uma Linha de Produtos de Software (LPS) para diferentes escolas sem alteração direta no código-fonte.

- **Complexidade Ciclomática (CC):**

Métodos de consulta como `consultarEmprestimos` possuem CC elevada devido à construção dinâmica de especificações (`EmprestimoSpecification`), dificultando testes unitários isolados e o reuso dessa lógica em outros contextos

2.1. Evidências Métricas

Fizemos algumas análises usando alguns scripts em python para fazer a coleta das métricas. Verificamos os arquivos e utilizamos as métricas da sessão acima para checar o sistema antes de começar a implementação das refatorações, os resultados prévios foram esses:

Classe	CBO (Couplin)	LCOM (Cohesion)	Complexida de (Avg CC)	Problema Identificado
PdfExportService	8	0	1.00	<i>God Class</i> (legado): acoplamento e responsabilidade centralizada.
AbstractPdfReport	11	9	1.00	<i>Template Method</i> rígido acoplado a biblioteca externa (OpenPDF).
PdfHeaderUtil	3	0	1.00	Baixa variabilidade (dados hardcoded).
FrequenciaAlunosPdfReport	5	0	1.50	Lógica de apresentação misturada com lógica de negócio.
FrequenciaAlunosPdfFactory	4	0	1.00	Factory leve, pouco acoplamento.
AlunosMaisLeitoresPdfReport	5	0	1.50	Report concreto (usava OpenPDF).
LivrosMaisLidosPdfReport	5	0	1.50	Report concreto (usava OpenPDF).
RelatorioAcervoPdfReport	5	0	1.50	Report de acervo com lógica de apresentação.

OcorrenciasPdfReport	5	0	1.50	Report concreto (usava OpenPDF).
TurmasMaisLeitorasPdfReport	5	0	1.50	Report concreto (usava OpenPDF).
EmprestimosService	14	3	2.80	Service com orquestração de regras e notificações (legado).
EmprestimoFacade	23	10	1.58	Facade: orquestra observadores e validators (legado).
NotificationService	1	0	1.33	Delegador para estratégias de notificação (legado).
EmailSend	0	0	1.00	Utilitário de envio SMTP (legado).
NotificationObserver	8	0	2.00	Observer com lógica de formatação e envio.

3. Melhorias Propostas

Com o objetivo de elevar o nível de reuso de "ad-hoc" para "sistemático", as seguintes melhorias foram planejadas:

Integração com API de Microserviço de PDF

A melhoria mais significativa consiste em substituir a renderização interna de PDFs (baseada na biblioteca OpenPDF/LibrePDF) pela integração com o PDF-Microservice.

- **Solução:** O PdfExportService deixará de gerar o documento binário localmente. Ele passará a atuar como um cliente que envia dados em formato JSON para a API do microserviço.
- **Impacto no Reuso:** Desacopla o layout (templates Handlebars) do código Java. Mudanças visuais nos relatórios não exigirão mais a recompilação do Bibliotech,

permitindo que o motor de PDF seja um Ativo Base compartilhado entre diferentes sistemas.

Padrão Strategy para Notificações

Atualmente, o sistema utiliza o EmailSend diretamente no fluxo de empréstimos.

- **Solução:** Implementar a interface NotificationStrategy e o NotificationService já esboçados no código.
- **Impacto no Reuso:** Permite adicionar novos canais (SMS, WhatsApp) como plugins, sem modificar a lógica central de empréstimos, seguindo os princípios de variabilidade de uma LPS.

Expansão do Padrão Facade

- **Solução:** Consolidar todas as interações complexas de persistência e validação dentro do EmprestimoFacade.
- **Impacto no Reuso:** Simplifica a interface do EmprestimosService, reduzindo seu CBO e tornando o componente de "negócio de empréstimo" mais fácil de ser reutilizado em uma eventual interface mobile ou via CLI.

4. Implementação

Notificações-Strategy

Foi implementada uma solução completa do Padrão Strategy para notificações. Antes o código acoplado diretamente ao *EmailSend*, era difícil adicionar novos canais de notificação e mudanças em um canal afetavam todo o sistema. Após a implementação temos múltiplos canais de notificação (Email, SMS, WhatsApp), ficou mais fácil a adição de novos canais como plugins e a mudança de canal não altera o código de negócio. Foi adaptado também o *EmprestimosService* para que ele utilizasse o novo modelo de notificação.

Segue abaixo dois exemplos de código, apenas para ilustrar o “rosto” dessa implementação.

EmailNotificationStrategy:

```
Component
public class EmailNotificationStrategy implements NotificationStrategy {
    private final EmailSend mailSender;

    @Autowired
    public EmailNotificationStrategy(EmailSend mailSender) {
        this.mailSender = mailSender;
    }

    @Override
    public boolean send(String recipient, String subject, String message) {
        mailSender.sendEmail(recipient, subject, message);
        return true;
    }
}

return go(f, seed, [])
}
```

WhatsAppNotificationStrategy:

```
@Component
public class WhatsAppNotificationStrategy implements NotificationStrategy {

    @Override
    public boolean send(String recipient, String subject, String message) {}

    private boolean isValidWhatsAppNumber(String phone) {}

    private String formatWhatsAppMessage(String subject, String message) {}
}
```

Empréstimo-Facade

Foi implementado o padrão estrutural Facade (EmprestimoFacade) para centralizar a orquestração de regras de negócio complexas e persistência, retirando essa responsabilidade do EmprestimosService. Juntamente a isso, aplicou-se o padrão Observer, permitindo que ações secundárias, como auditoria de logs e cálculo de estatísticas, sejam disparadas através de eventos (EmprestimoEvent) de forma assíncrona e independente, removendo a necessidade de chamadas diretas dentro do fluxo crítico de criação ou atualização de empréstimos.

Com isso, houve uma redução significativa do acoplamento no serviço principal, que passou de um gestor de múltiplas dependências para um orquestrador leve, e um aumento da extensibilidade do sistema. O código ficou mais limpo e seguro para manutenção, pois novas funcionalidades (como notificações por e-mail) podem ser adicionadas criando-se novos observers sem a necessidade de alterar ou arriscar a integridade da lógica central de empréstimos, respeitando o princípio Aberto/Fechado (OCP) do SOLID.

Orquestração Linear (Método realizarEmprestimo)

```
@Transactional
public EmprestimoResponseDTO realizarEmprestimo(EmprestimoRequestDTO requestDTO) {
    Aluno aluno = alunoValidator.validarParaEmprestimo(requestDTO.getIdAluno());
    Exemplar exemplar = exemplarValidator.validarDisponibilidade(requestDTO.getIdExemplar());
    Usuario usuario = obterUsuarioAtual();

    Emprestimo emprestimo = criarEmprestimoEntidade(requestDTO, aluno, exemplar, usuario);

    alunoValidator.marcarComoDebito(aluno);
    exemplarValidator.marcarComoEmprestado(exemplar);

    Emprestimo emprestimoSalvo = emprestimoRepository.save(emprestimo);

    notifyObservers(new EmprestimoEvent(emprestimoSalvo, EventType.EMPRESTIMO_CRIADO));

    return emprestimoResponseMapper.toDto(emprestimoSalvo);
}
```

Redução de Acoplamento no Service (O Cliente)

```
@Service
public class EmprestimosService {
    private final EmprestimoFacade emprestimoFacade;
    private final EmprestimoRepository emprestimoRepository;
    @Override
    public EmprestimoResponseDTO save(EmprestimoRequestDTO emprestimoRequestDTO) {
        return emprestimoFacade.realizarEmprestimo(emprestimoRequestDTO);
    }

    @Override
    public String update(Integer id, EmprestimoRequestDTOConcluir dto) {
        return emprestimoFacade.concluirEmprestimo(id, dto);
    }
}
```

Encapsulamento de Regras (Os Subsistemas)

```
public Aluno validarParaEmprestimo(Integer id) {
    Aluno aluno = repository.findById(id).orElseThrow(...);
    if (!aluno.isAtivo()) throw new ValidationException("Aluno inativo");
    if (aluno.possuiMultas()) throw new ValidationException("Aluno com multas");
    if (aluno.limiteEmprestimosAtingido()) throw new ValidationException("Limite excedido");
    return aluno;
}
```

Desacoplamento de Efeitos Colaterais (Observer Pattern)

```
@Override
public void notifyObservers(EmprestimoEvent event) {
    for (EmprestimoObserver observer : observers) {
        try {
            observer.update(event);
        } catch (Exception e) {
            logger.error("Erro no observer: {}", e.getMessage());
        }
    }
}
```

Refatoração de Subsistema de Relatórios (PDF)

A renderização dos pdf foi externalizada, passando agora a responsabilidade da geração para o pdf-microservice. O PdfExportService agora utiliza estratégias que implementam PdfExportStrategy para converter entidades JPA em DTOs, eliminando a dependência da biblioteca OpenPDF.

Interface Unificada:


```

public interface PdfExportStrategy {
    // Contrato agnóstico à implementação
    byte[] exportar(DadosRelatorio dados);
    String getStrategyName();
}

```

Integração com Microserviço de PDF

Foi feita a integração com o microserviço e implementados padrões de resiliência para melhorar a qualidade da integração. O PdfMicroserviceClient (Que consome a api externa) foi desenvolvido utilizando Spring WebClient, que é uma biblioteca que otimiza as requisições e utiliza técnicas para tornar elas não bloqueantes, o que permite interações externas assíncronas no Java, o que é muito importante para um sistema que utiliza a arquitetura distribuída. Seguindo as restrições técnicas do projeto, foram aplicadas políticas de resiliência para garantir robustez em ambiente distribuído:

Timeout: 30 segundos (evita bloqueio de threads). Retry: 3 tentativas com Exponential Backoff (apenas para erros 5xx ou falhas de rede).

Classe do Client:

```

@Component
public class PdfMicroserviceClient {
    private static final Logger log =
    LoggerFactory.getLogger(PdfMicroserviceClient.class);

    private final WebClient webClient;
    private final Duration timeout;
    private final int retryAttempts;

    public PdfMicroserviceClient(WebClient.Builder webClientBuilder,
                                @Value("${pdf.microservice.base-
url:http://localhost:3000}") String baseUrl,
                                @Value("${pdf.microservice.timeout-
seconds:30}") long timeoutSeconds,
                                @Value("${pdf.microservice.retry-
attempts:3}") int retryAttempts) {
        this.webClient = webClientBuilder.baseUrl(baseUrl).build();
        this.timeout = Duration.ofSeconds(timeoutSeconds);
        this.retryAttempts = retryAttempts;
    }
}

```

Uso da Resiliência na integração:

```
// Exemplo de configuração de resiliência no cliente
Retry retryPolicy = Retry.backoff(3, Duration.ofSeconds(2))
    .filter(throwable -> throwable instanceof IOException || throwable
        instanceof WebClientResponseException.InternalServerError);
```

5. Resultados

Comparativo de Métricas (Antes vs. Depois)

A refatoração resultou em melhorias quantitativas significativas na qualidade do código, trouxe algum aumento de acoplamento, por exemplo por conta da chamada para o microserviço, por conta das importações, mas em geral teve um resultado interessante, reduzindo CBO ou LCOM.

Métricas do Padrão Strategy (Notificações)

Componente	CBO (Antes → Depois)	LCOM (Antes → Depois)	Observação
NotificationService	1 → 1	0 → 0	Delegação para estratégias; mantém baixa complexidade.
EmailSend	0 → 0	0 → 0	Utilitário estável.
EmailNotificationStrategy	N/A → 2	N/A → 0	Nova estratégia (aplicação do Strategy).
SMSNotificationStrategy	N/A → 3	N/A → 0	Nova estratégia; validação/truncamento.
WhatsAppNotificationStrategy	N/A → 3	N/A → 1	Nova estratégia; validação/format.
NotificationObserver	8 → 8	0 → 0	Observador manteve responsabilidades; alvo para futura extração.

Métricas do Padrão Facade (Empréstimos)

Componente	CBO (Antes → Depois)	LCOM (Antes → Depois)	Observação
EmprestimosService	14 → 13	3 → 3	Responsabilidades isoladas; leve queda de acoplamento.
EmprestimoFacade	23 → 23	10 → 10	Orquestra validações e observers; CBO alto por natureza.
EmprestimoController	10 → 10	0 → 0	Interface estável; sem mudanças métricas.

Métricas da Refatoração do Subsistema de Relatórios (PDF)

Componente	CBO (Antes → Depois)	LCOM (Antes → Depois)	Observação
PdfExportService	8 → 17	0 → 0	De monolito a delegador: LOC e CBO aumentam por composição/cliente.
AbstractPdfReport	11 → 11	9 → 9	Template mantido; coesão alta.
PdfHeaderUtil	3 → 3	0 → 0	Dados institucionais devem ser externalizados.
FrequenciaAlunosPdfReport	5 → 5	0 → 0	Reportes concretos sem mudanças nas métricas.

Integração com PDF-Microservice

Componente	CBO (Antes → Depois)	LCOM (Antes → Depois)	Observação
PdfMicroserviceClient	N/A → 4	N/A → 0	Novo cliente HTTP, responsabilidade única.
PdfRequest	N/A → 0	N/A → 0	DTO leve; baixa complexidade.

Resumo: A média do sistema refletiu maior modularidade e testabilidade — a reorganização reduziu dependências explícitas e deixou componentes com responsabilidade mais clara, mesmo que aumentando o valor de algumas métricas como consequência.

6. Discussão

Barreiras técnicas, culturais ou organizacionais percebidas

Tecnicamente, mesmo que o sistema não seja considerado "legado", houve uma barreira inicial de readaptação. Como houve uma pausa no desenvolvimento, a equipe precisou investir um tempo considerável apenas para entender novamente o código e a estrutura do projeto antes de aplicar as mudanças. Ainda no aspecto técnico, houve dificuldade com as **métricas de reuso**. Por ser um conteúdo bem específico, e que a equipe não via desde o ano passado, surgiram dúvidas conceituais durante o processo. Para não travar o andamento, a equipe optou por seguir adiante aplicando as métricas conforme a interpretação que julgou mais correta para o contexto.

No aspecto organizacional, o maior desafio foi o **calendário apertado**. O fim do semestre trouxe uma grande sobrecarga de tarefas de outras disciplinas. Isso gerou um conflito de prioridades, obrigando os membros da equipe a fazerem horas extras e intensificarem o ritmo no período final para conseguir entregar tudo dentro do prazo.

Estimativa de custos/benefícios do reuso aplicado

O principal "custo" do processo foi o desgaste e o tempo extra investido pela equipe para lidar com a pressão do cronograma e as dúvidas técnicas.

No entanto, os benefícios compensaram esse esforço. O projeto, que estava parado antes da disciplina de Reuso de Software, foi retomado e recebeu melhorias significativas em sua estrutura. Além da evolução do software em si, houve um grande ganho no **aprendizado da equipe**, que pôde aplicar na prática os conceitos estudados e evoluir tecnicamente.

7. Conclusão

A reestruturação arquitetural desacoplou regras de negócio da infraestrutura, transformando componentes monolíticos em módulos testáveis. O uso dos padrões *Strategy* e *Facade*, aliado à migração para microsserviços, reduziu significativamente o acoplamento e eliminou código redundante, isolando funcionalidades como notificações e relatórios do núcleo da aplicação.

As métricas coletadas confirmam a eficácia da transição do Biblitech para um modelo de reuso sistemático. No *EmprestimosService*, a aplicação do *Facade* limpou a orquestração de regras e reduziu o CBO de 14 para 13. Já no subsistema de relatórios, o *PdfExportService* apresentou um

aumento no CBO (de 8 para 17), um trade-off necessário para transformar uma "God Class" em uma delegadora eficiente, permitindo a integração com o microserviço de PDF.

Apesar da cobertura de testes limitada pelo prazo, a arquitetura provou-se evolutiva através da notificação plugável e da externalização de dados. O projeto entrega uma base robusta para uma Linha de Produtos de Software (LPS), compensando a complexidade pontual das classes delegadoras com maior modularidade e manutenibilidade.

8. Referências

- Slides da Disciplina
- [Padrão Strategy - Gang of Four](#)
- [Linha de Produto de Software](#)
- [SOLID Principles](#)

9. Anexos

Relatório de Acervo

Título	Autor	Quantidade
Livro Script 1 20260116044126	Autor Script	2
Livro Script 1 20260116044415	Autor Script	2
Livro Script 2 20260116044127	Autor Script	2
Livro Script 2 20260116044415	Autor Script	2
Livro Script 3 20260116044127	Autor Script	2
Livro Script 3 20260116044416	Autor Script	2

Ocorrências

Aluno	Registrada Por	Detalhes	Data
Aluno Token 20260116032542	Admin Exemplo	Ocorrencia via token 20260116032542	16/01/2026
Aluno Token 20260116033055	Admin Exemplo	Ocorrencia via token 20260116033055	16/01/2026
Aluno Token 20260116034952	Admin Exemplo	Ocorrencia via token 20260116034952	16/01/2026
Aluno Token 20260116035052	Admin Exemplo	Ocorrencia via token 20260116035052	16/01/2026
Aluno Token 20260116035118	Admin Exemplo	Ocorrencia via token 20260116035118	16/01/2026
Aluno Token 20260116040809	Admin Exemplo	Ocorrencia via token 20260116040809	16/01/2026